Universidad Da Vinci de Guatemala

Facultad de Ingenieria, Industria y Tecnologia

Desarrollo de Aplicaciones Moviles (Virtual)

Evaluacion Parcial II

Objetivo general

Evaluar la capacidad del estudiante para disenar e implementar una aplicacion Android utilizando Kotlin, XML Layouts (LinearLayout y ConstraintLayout), consumo de API REST, y el patron de arquitectura MVVM, mediante el desarrollo de una aplicacion de gestion de notas personales.

Serie Unica: Aplicacion Practica (100 puntos)

Descripcion del proyecto

El estudiante debera crear un proyecto Android nuevo desde cero llamado NotasApp que implemente un sistema de notas personales. La aplicacion debe permitir a un usuario autenticado crear notas con un titulo y un contenido, almacenarlas en el backend, y consultar el listado de todas sus notas guardadas. La aplicacion debe contar con un login, un dashboard y dos funcionalidades principales: creacion de notas y visualizacion de notas.

Lineamientos tecnicos obligatorios

- Lenguaje: Kotlin
- UI: XML Layouts (LinearLayout y ConstraintLayout segun se indique en cada pantalla)
- Arquitectura: MVVM (Model - View - ViewModel)
- Estado: LiveData
- Consumo de API: Retrofit con OkHttp
- Backend: Conectado al backend del estudiante (el mismo que han venido trabajando)

1. Login (20 puntos)

La aplicacion debe contar con una pantalla de inicio de sesion que autentique al usuario contra el backend.

a) Implementar la pantalla de login utilizando ConstraintLayout como layout raiz.

b) La pantalla debe contener campos para usuario/email y contrasena, y un boton para iniciar sesion.

c) La autenticacion debe realizarse consumiendo el endpoint correspondiente del backend. No se permite login hardcodeado ni datos mock.

d) Al autenticarse exitosamente, el backend retornara un token. Dicho token debe almacenarse y enviarse automaticamente en el header de autorizacion de todas las peticiones subsiguientes al backend.

e) Mostrar mensajes de error al usuario cuando las credenciales sean incorrectas o exista un error de conexion.

f) Tras login exitoso, navegar al Dashboard.

g) Seguir el patron MVVM: ViewModel con LiveData, Repository y ApiService separados.

2. Dashboard (10 puntos)

La aplicacion debe contar con una pantalla principal que sirva como punto de acceso a las funcionalidades.

a) Implementar la pantalla del Dashboard utilizando LinearLayout como layout raiz.

b) Mostrar un mensaje de bienvenida que incluya el nombre del usuario autenticado.

c) Incluir dos botones claramente identificados: uno para navegar a la pantalla de creacion de notas y otro para navegar a la pantalla de listado de notas.

3. Creacion de notas (25 puntos)

La aplicacion debe contar con una pantalla que permita al usuario crear una nueva nota y guardarla en el backend.

a) Implementar la pantalla de creacion utilizando ConstraintLayout como layout raiz.

b) La pantalla debe contener campos para el titulo y el contenido de la nota.

c) Al presionar el boton de guardar, enviar los datos al backend mediante el endpoint correspondiente.

d) Mostrar un indicador de carga durante el envio.

e) Mostrar un mensaje de exito o error segun la respuesta del backend.

f) Seguir el patron MVVM completo: State/LiveData, ViewModel, Repository.

4. Listado de notas (25 puntos)

La aplicacion debe contar con una pantalla que muestre todas las notas guardadas por el usuario, obtenidas desde el backend.

a) Implementar la pantalla de listado utilizando LinearLayout como layout raiz.

b) Consumir el endpoint correspondiente del backend para obtener las notas del usuario autenticado.

c) Mostrar las notas en un RecyclerView donde cada elemento presente el titulo y el contenido de la nota.

d) Cada elemento del RecyclerView debe utilizar un CardView con ConstraintLayout como layout interno.

e) Mostrar un indicador de carga mientras se obtienen los datos.

f) Seguir el patron MVVM completo.

5. Desarrollo de endpoints en el backend (20 puntos)

El estudiante debe desarrollar en su backend los endpoints necesarios para soportar todas las funcionalidades de la aplicacion movil.

a) Endpoint para crear una nota: debe recibir titulo y contenido, y almacenarlos en la base de datos asociados al usuario autenticado.

b) Endpoint para obtener las notas: debe retornar todas las notas del usuario autenticado.

c) Ambos endpoints deben validar el token de autenticacion.

d) La tabla de notas en la base de datos debe contener como minimo los campos necesarios para almacenar la relacion con el usuario, el titulo, el contenido y la fecha de creacion.

Rubrica de evaluacion

Criterio	Excelente	Bueno	Adecuado	Deficiente	Pts
Login	Autenticacion contra backend, token en headers, manejo de errores, MVVM, ConstraintLayout	Login funciona pero sin interceptor de token o manejo de errores incompleto	Login parcialmente funcional o con datos mock	No funcional o no implementado	**20**
Dashboard	LinearLayout, bienvenida con nombre del usuario, navegacion a ambas funcionalidades	Funcional con fallos menores en layout o navegacion	Dashboard parcial sin datos del usuario o navegacion incompleta	No implementado	**10**
Creacion de notas	Formulario completo, envio al backend, carga/error, MVVM, ConstraintLayout	Funcional con fallos menores en estructura o estados	Formulario no envia al backend o estructura incompleta	No implementado o no funcional	**25**
Listado de notas	RecyclerView con CardView, datos del backend, carga, MVVM, LinearLayout	Funcional con fallos menores en diseno o estructura	Listado parcial, datos locales o sin backend	No implementado	**25**
Backend endpoints	Ambos endpoints funcionales, auth por token, persistencia correcta, tabla completa	Endpoints funcionales con fallos menores en validacion	Un solo endpoint o sin validacion de token	No implementado	**20**
				Total	**100**